

H31 専門科目 III. 問(3) — 詳細トレース (補足解説)

操作列： put(7) put(3) put(1) put(9) put(3) get() put(4) put(6) put(8) get()

最終解：get()=3, size=6, x={3,4,6,9,7,8,8,0,0}

この構造は「最小ヒープ (min-heap)」

up が while(x[親] > x[自分]) で交換 → 小さい値が上に昇る → **根が常に最小値。**

配列と木の添字対応 (0始まり):

親 = $(i-1)/2$ 左の子 = $2i+1$ 右の子 = $2i+2$ (/ は整数の切り捨て)

i:	1	2	3	4	5	6	
親:	0	0	1	1	2	2	$\leftarrow (2-1)/2=0, (4-1)/2=1, (6-1)/2=2$

1. put フェーズ (昇る = up)

put は「末尾に置く → 親より小さければ交換して昇る」。比べる相手は **木の親 (i-1)/2** (配列のとなり i-1 ではない)。

put(7) → put(3) → put(1)

```
put(7): x=[7]
```

put(3): $x[1]=3$, 親 $(1-1)/2=0$, $x[0]=7>3 \rightarrow$ 交換 3
 $x=[3,7]$

7

put(1): $x[2]=1$, 親 $(2-1)/2=0$, $x[0]=3>1 \rightarrow$ 交換
 $x=[1,7,3]$

```

1      ← x[2]の親は x[0] !
    / \      (x[1]ではない)
   7   3

```

つまりポイント： put(1) で「x[2] の親は x[1]」と誤ると [1,3,7] になる。正しくは $(2-1)/2=0$ なので親は **x[0]**、結果は [1,7,3]。

put(9) → put(3)

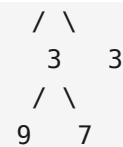
```
put(9): x[3]=9, 親(3-1)/2=1, x[1]=7>9? no →動かず
x=[1,7,3,9]
```

$$\begin{array}{c} 1 \\ \quad / \quad \backslash \\ \quad 7 \quad 3 \\ \quad / \\ 9 \end{array}$$

put(3): $x[4]=3$, 親 $(4-1)/2=1$, $x[1]=7>3 \rightarrow$ 交換

1

i=1, 親0, x[0]=1>3? no →停止
x=[1,3,3,9,7]



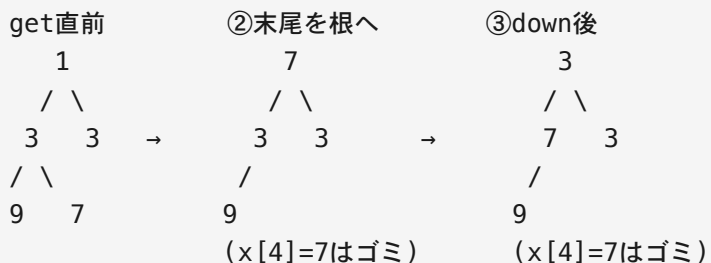
1回目 get の直前: x=[1,3,3,9,7], size=5 (根は最初に入れた7ではなく 1)

2. 1回目の get() (沈む = down)

get の3ステップ: ① 根を控える ② size を1減らし末尾を根へ ③ down で沈める

```
int get(){
    result = x[0];          // ① 根 (最小値) を控える
    x[0] = x[--size];       // ② size 5→4, 末尾 x[4] を根へコピー
    down(0, size);          // ③ down(0,4) で沈める
    return result;          // ④ 控えた値を返す
}
```

ステップ	動作	配列の状態
① result	x[0]=1 を控える (戻り値)	[1,3,3,9,7]
② x[0]=x[--size]	size 5→4, x[0]=x[4]=7	[7,3,3,9,7]
③ down(0,4)	根7と小さい子3を交換	[3,7,3,9,7]



get()=1, size=4, x=[3,7,3,9,7,0,0,0,0] ※ x[4]=7 は「末尾→根」のコピー元が消されず残ったゴミ

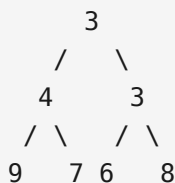
3. put(4) → put(6) → put(8)

put(4): x[4]=4(ゴミ7を上書き), size=5
up(4): 親1, x[1]=7>4 →交換, i=1, 親0, x[0]=3>4? no
x=[3,4,3,9,7]

put(6): x[5]=6, size=6
up(5): 親(5-1)/2=2, x[2]=3>6? no →停止
x=[3,4,3,9,7,6]

put(8): x[6]=8, size=7
up(6): 親(6-1)/2=2, x[2]=3>8? no →停止
x=[3,4,3,9,7,6,8]

2回目 get の直前: x=[3,4,3,9,7,6,8,0,0], size=7



4. 2回目の get() — 詳細

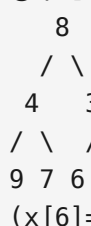
① result=x[0]=3 (戻り値) ② size 7→6, x[0]=x[6]=8 (x[6]=8 はゴミに) ③ down(0,6)

down(0,6) の中身

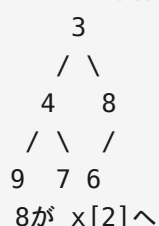
down のルール: 左右の子のうち小さいほうを選び、x[親] > x[小さい子] なら交換 (最小ヒープなので小さい子を上げる)。

呼び出し	i	子の比較	判定	結果
down(0,6)	0	x[1]=4, x[2]=3 → 小=x[2]	x[0]=8>3 →交換	[3,4,8,9,7,6]
down(2,6)	2	x[5]=6のみ(x[6]は範囲外)	x[2]=8>6 →交換	[3,4,6,9,7,8]
down(5,6)	5	c=2·5+1=11 ≥ 6	子なし →return	停止

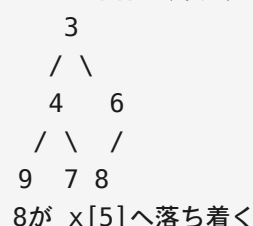
②末尾8を根へ



down 1回目



down 2回目 (確定)



「8」が2つ現れる理由：put(8)で入れた**正規の8**は②→③で根から x[5] まで沈んだもの。一方 **x[6]=8** は②で「コピー元」になっただけで上書きされず残った**ゴミ**。同じ値なので紛らわしいが別物。

5. 最終結果

get()=3, size=6, x={3,4,6,9,7,8,8,0,0}

```
x = [3, 4, 6, 9, 7, 8, 8, 0, 0]    size=6
      0  1  2  3  4  5  6
                ↑ ゴミの8 (size外, x[6])
```

有効ヒープ(6個): 3≤4,6 ✓ 4≤9,7 ✓ 6≤8 ✓

```
      3
     / \
    4   6
   / \ /
  9  7 8
```

まとめ (要点)

- ・最小ヒープ：up が「親>自分で交換」→ 根が常に最小 (get は最小値を返す)
- ・添字：親=(i-1)/2 (切り捨て) , 子=2i+1, 2i+2
- ・put：末尾に置いて親と比べ昇る (up)
- ・get：①根を控える ②size--して末尾を根へ ③down で沈める
- ・down：左右の子の小さいほうと比べ、親が大きければ交換
- ・配列は0クリアされない → size外に古い値 (ゴミ) が残る (x[6]=8 の正体)
- ・答え：get()=3, size=6, x={3,4,6,9,7,8,8,0,0}

補足解説 (問3トレース) / 本資料は既存解答 2018_解答_3.pdf の (3) を手順分解したもの。